Project-HLD

On

Multimodal Manufacturing Creator

Course Name: Generative AI

Institution Name:Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Pooja Dangi	EN22CS301708
2	Pransu Singh	EN22CS301735
3	Pratik Patel	EN22Cs301748
4	Rahul Negi	EN22CS301778
5	Rupesh Sirvi	EN22CS301834

Group Name:09D7

Project Number:GAI-32

Industry Mentor Name:

University Mentor Name:Dr.Hemlata Patel

Academic Year:2025-2026

Table of Contents

1 . Introduction.

1. Scope of the document.
2. Intended Audience
3. System overview.

2. System Design.
1. Application Design
2. Process Flow.
3. Information Flow.
4. Components Design
5. Key Design Considerations
6. API Catalogue.

3. Data Design.
1. Data Model
2. Data Access Mechanism
3. Data Retention Policies
4. Data Migration

4. Interfaces
5. State and Session Management
6. Caching
7. Non-Functional Requirements
1. Security Aspects
2. Performance Aspects

8. References

1. Introduction

The **Multimodal Manufacturing Creator** is a Generative AI-based system that transforms textual manufacturing ideas into **comprehensive outputs**, including:

- Detailed product descriptions
- Functional explanations
- High-quality visual prototypes

This system integrates:

- **Large Language Models (LLMs)** → for text generation
- **Image Generation Models (via API)** → for visualization
- **Vector Database** → for semantic retrieval and context enhancement

The goal is to enable users to quickly **visualize, understand, and present manufacturing concepts** without needing complex design tools like CAD software.

1.1 Scope of the Document

This document provides a detailed low-level design of the Multimodal Manufacturing Creator, including:

- Multimodal AI workflow (text + image generation)
- Integration of LLM APIs and Image APIs
- Prompt processing and enhancement
- System architecture and execution flow

- Backend services and database design

The scope also includes implementation details such as:

- Prompt engineering pipeline
- Vector database usage for context retrieval
- API structure for generation modules
- Storage of generated outputs

1.2 Intended Audience
This document is intended for:

- Software Developers
- AI/ML Engineers
- System Designers
- Project Evaluators

- Students working on Generative AI projects

1.3 System Overview
The system allows users to:

- Input manufacturing ideas (e.g., *"Eco-friendly smart water bottle with temperature control"*)
- Automatically generate:
  - Detailed product descriptions
  - Design specifications
  - Realistic product images

Key Features

- Multimodal generation (text + image)
- Prompt enhancement using LLM
- Semantic search using Vector DB
- Real-time visualization
- Download/export outputs

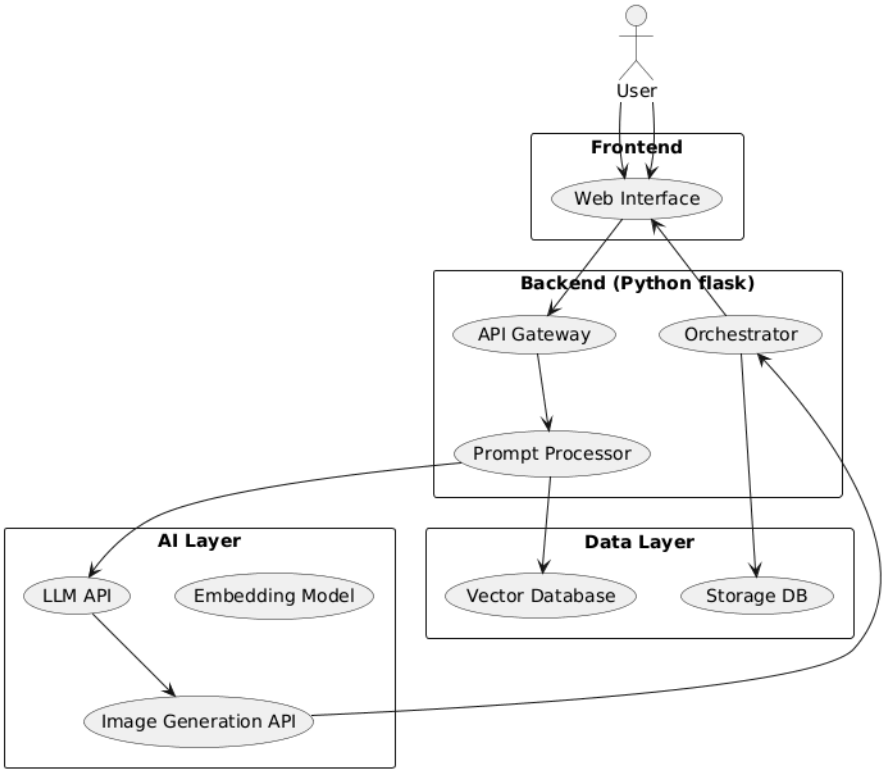
2. System Design

2.1 Application Design

Architecture Type

Client-Server Architecture with AI Integration

SYSTEM ARCHITECTURE:



System Layers

1. Frontend Layer

- Web interface (React / HTML / JS)
- Input field for prompts
- Output display (text + images)
- User dashboard

2. Backend Layer (Python)

- Handles business logic
- API orchestration
- Prompt processing

3. AI Layer

- LLM API (text generation)

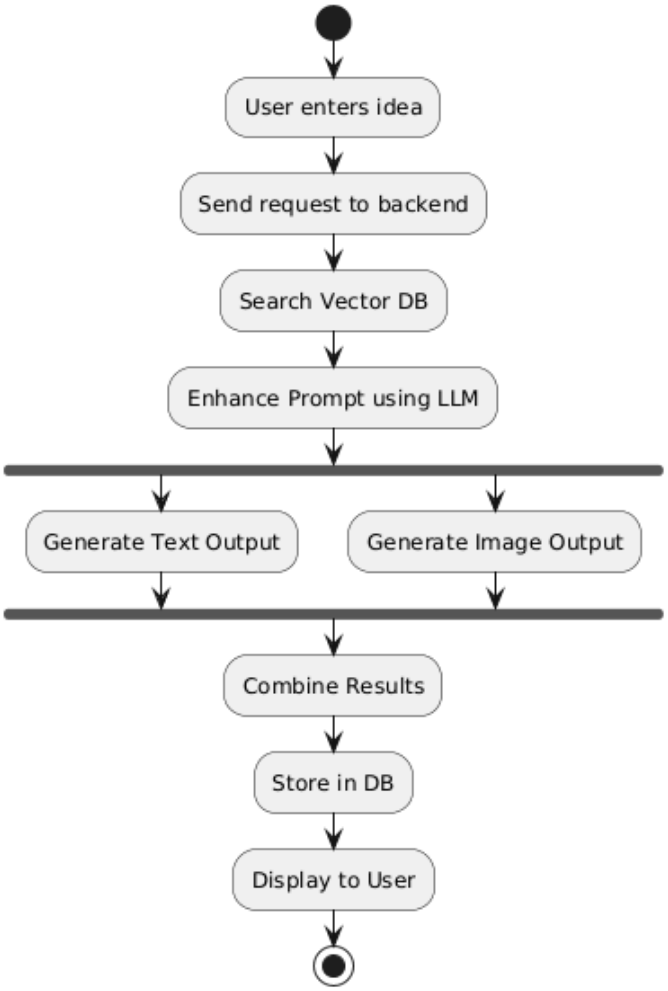
- Image Generation API
- Prompt Engineering Module

4. Data Layer

- Vector Database (semantic retrieval)
- Storage database (user data, outputs)

2.2 Process Flow

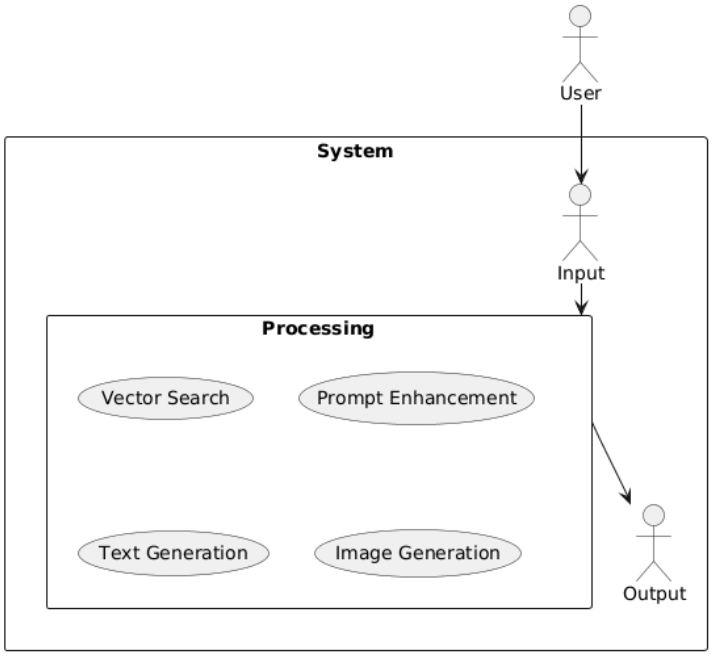
PROCESS FLOW DIAGRAM:



1. User logs into system
2. User enters manufacturing concept
3. Backend receives input
4. Query sent to Vector DB:
    - Retrieve related manufacturing knowledge
5. Prompt is enhanced using LLM

6. Enhanced prompt is used for:
    - Text generation (LLM)
    - Image generation (API)
7. Results are returned to backend
8. System displays:
    - Description
    - Generated image
9. Output stored in database

2.3 Information Flow



Input

- User prompt
- Optional parameters (style, material, industry)

Processing

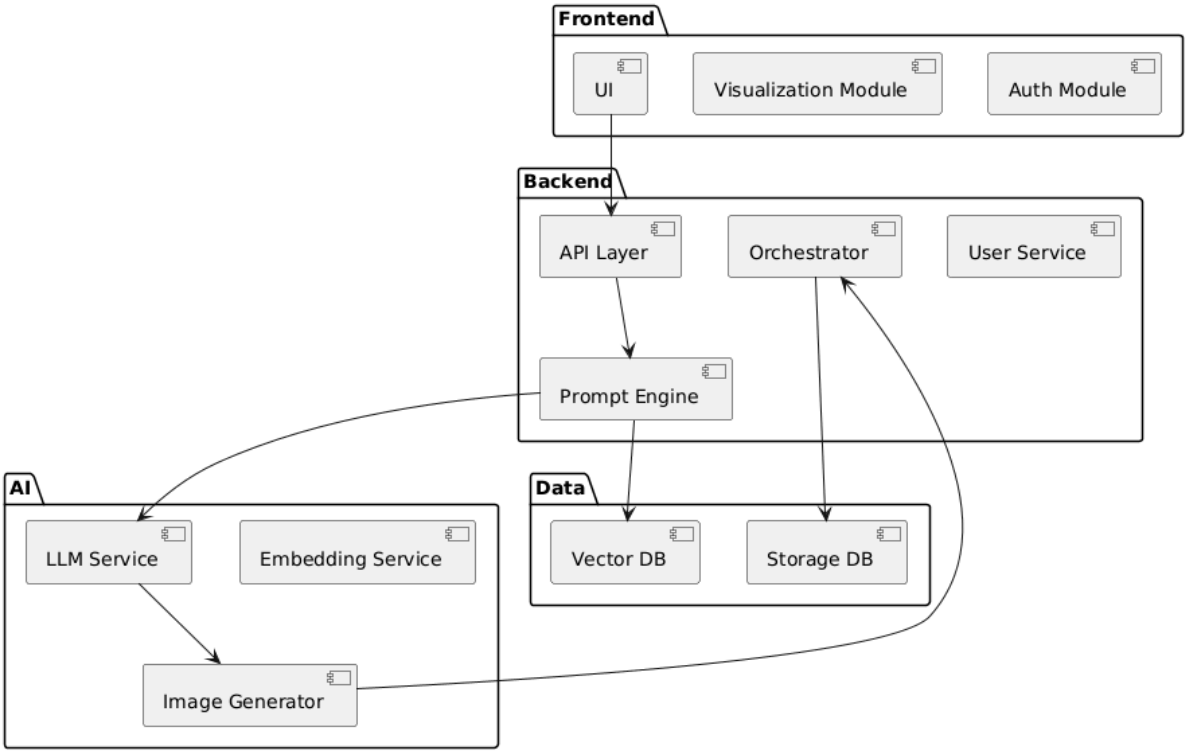
- Semantic search via Vector DB
- Prompt enhancement using LLM
- Parallel generation:
    - Text generation
    - Image generation

Output

- Product description
- Functional explanation

- Generated prototype image
- Suggestions/improvements

2.4 Components Design



1. Frontend Components

a. User Interface (UI)

- Prompt input
- Output display
- Navigation dashboard

b. Visualization Module

- Displays generated images
- Zoom / download options

c. Input Control Module

- Accepts structured/unstructured prompts
- Allows parameter selection

d. Authentication Module

- Login / Register

- User session management

2. Backend Components (Python)

a. API Layer

- Receives frontend requests
- Routes to services

b. Prompt Engineering Module

- Cleans and structures input
- Enhances prompt using LLM
- Combines Vector DB context

c. Vector Search Module

- Uses embeddings
- Retrieves relevant manufacturing knowledge
- Improves prompt quality

d. LLM Module

- Generates:
  - Product description
  - Design explanation
  - Use-case details

e. Image Generation Module

- Sends prompt to image API
- Generates prototype visuals

f. Multimodal Orchestrator

- Synchronizes text and image outputs
- Ensures consistency

g. Storage Module

- Stores:
  - Prompts
  - Generated text
  - Image URLs

h. User Management Service

- Handles authentication
- Stores user history

2.5 Key Design Considerations

1. Accuracy

- Use Vector DB + LLM for better context
- Reduce hallucinations

2. Scalability

- Modular microservice design
- Easy integration with new AI models

3. Performance

- Parallel API calls (text + image)
- Caching frequently used prompts

4. Security

- Secure API keys
- User authentication

5. User Experience

- Fast response time
- Clean UI
- High-quality visuals

6. Cost Optimization

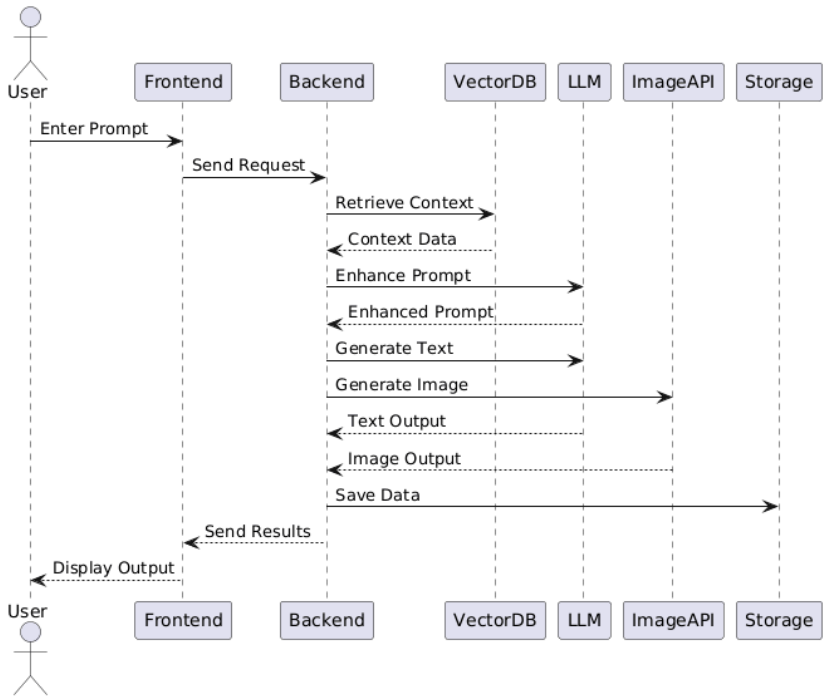
- Limit API calls

- Efficient prompt engineering

2.6 API Catalogue

API	Endpoint	Description
Prompt Enhancement	`/enhancePrompt`	Improves input
Text Generation	`/generateText`	Generates description
Image Generation	`/generateImage`	Generates visuals
Multimodal API	`/generateAll`	Full output
Vector Search	`/searchContext`	Retrieves context
User APIs	`/login`, `/register`	Auth

SEQUENCE DIAGRAM



3. Data Design

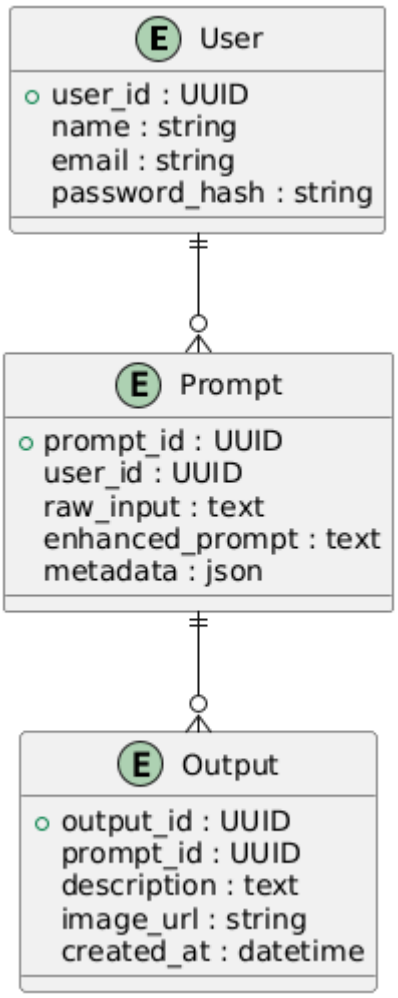
3.1 Data Model

The system uses a **hybrid data architecture**:

- **Relational/NoSQL DB** → structured storage (users, prompts, outputs)
- **Vector Database** → semantic search & context retrieval

Core Entities

- **User**
  - user_id (PK)
  - name, email, password_hash
  - created_at
- **Prompt**
  - prompt_id (PK)
  - user_id (FK)
  - raw_input
  - enhanced_prompt
  - metadata (industry, style, etc.)
- **Output**
  - output_id (PK)
  - prompt_id (FK)
  - description
  - image_url
  - created_at
- **Embedding (Vector DB)**
  - embedding_id
  - vector
  - source_text
  - metadata (domain, category)



3.2 Data Access Mechanism

Access Patterns

1. **User Flow**
   - Login → fetch user profile
   - Dashboard → fetch previous prompts
2. **Generation Flow**
   - Insert prompt
   - Fetch context (Vector DB)
   - Store results

API-Based Access

- REST APIs (JSON)

3.3 Data Retention Policies

Storage Strategy

- Store:
  - Prompts
  - Generated outputs
  - Image URLs (not heavy files)

Retention Rules

- User history: persistent
- Logs: temporary (7–30 days)
- Cache: short-lived (minutes/hours)

Optional Enhancements

- GDPR-style deletion:
  - User can delete history
- Archival storage:
  - Move old data to cold storage

3.4 Data Migration

When Needed

- Schema updates
- Model upgrades
- DB migration (local → cloud)

Strategy

- Versioned schema
- Backward compatibility
- Migration scripts

Example

- Add new field `style` to Prompt:
  - Default value applied to old records

4. Interfaces (Detailed)

Interfaces define **how components communicate internally and externally**.

4.1 User Interface (Frontend)

Features

- Prompt input field
- Output display (text + images)
- Dashboard (history)

Technologies

- React / HTML / JS

4.2 Backend API Interface

API Design Style

- RESTful APIs
- JSON-based communication

Core APIs

1. Prompt Enhancement

POST /enhancePrompt

2. Text Generation

POST /generateText

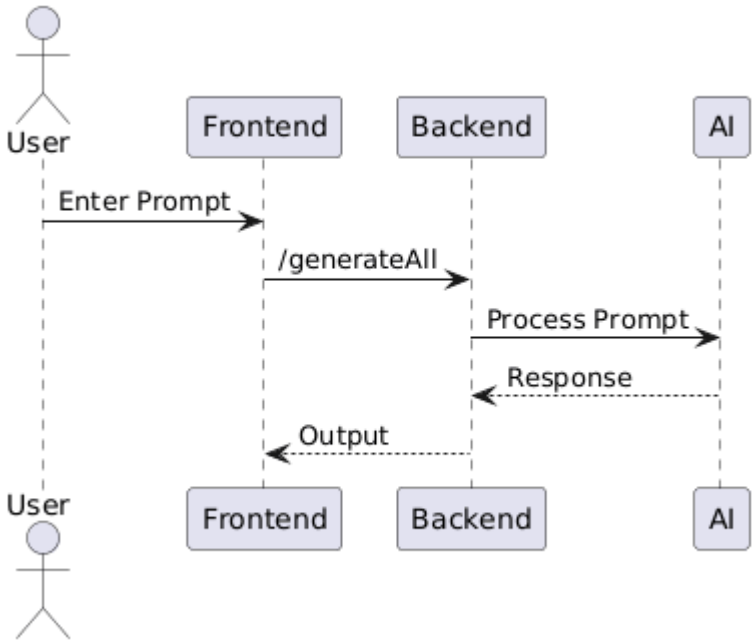
3. Image Generation

POST /generateImage

4. Multimodal API (Main)

POST /generateAll

API Interaction



4.3 External Interfaces

1. LLM API

- Used for:
  - Prompt enhancement
  - Text generation

2. Image Generation API

- Used for:
  - Prototype image generation

3. Vector Database API

- Used for:
  - Semantic search

4.4 Internal Interfaces

Between Components

- API Layer ↔ Prompt Engine

- Prompt Engine ↔ Vector DB
- Orchestrator ↔ AI Services

4.5 Interface Contracts

Each API follows:

- Request validation
- Standard response format

5. State and Session Management (Detailed)

This defines **how user identity and session continuity are handled**.

5.1 Session Strategy

Stateless Backend

- No session stored on server
- Each request carries authentication

5.2 Authentication Mechanism

Firebase Authentication

Flow:

1. User logs in
2. Server generates JWT
3. Token sent with every request

6. Caching

- Frequently used prompts cached
- API response caching
- Redis (optional)

7. Non-Functional Requirements

7.1 Security

- HTTPS
- Token-based authentication
- Secure API keys

7.2 Performance

- Parallel API execution
- Low latency (<2–5 sec target)